

```
; =====  
; HEDEF SAYIYA ULAS - 8086 Assembly Oyunu  
; Ogresci: Serra Tanis No: 170424046  
; EMU8086 COM formati - 8086 uyumlu  
; =====
```

```
#make_COM#  
ORG 100h
```

```
JMP START
```

```
NUM_OBJ EQU 10  
FIELD_RIGHT EQU 76  
FIELD_LEFT EQU 1  
TIMER_INIT EQU 80
```

```
; =====  
START:  
    MOV AH, 00h  
    MOV AL, 03h  
    INT 10h  
  
    MOV AX, 00h  
    INT 33h  
    MOV AX, 01h  
    INT 33h  
  
    CALL SHOW_HELP  
  
    MOV AH, 08h  
    INT 21h  
  
    CALL CLEAR_SCR  
    CALL DRAW_STATIC  
    CALL INIT_GAME  
    CALL DRAW_ALL_NUMS
```

```
; =====  
; ANA DONGU  
; =====
```

```
ANA_DONGU:  
    MOV AH, 0Bh  
    INT 21h  
    CMP AL, 0  
    JE FARE_KONTROL
```

```
    MOV AH, 08h  
    INT 21h  
    CMP AL, 'q'  
    JE FORCE_END  
    CMP AL, 'Q'  
    JE FORCE_END  
    CMP AL, ' '  
    JE KLAVYE_SECIM
```

```
FARE_KONTROL:  
    MOV AX, 03h  
    INT 33h
```

```
    TEST BX, 1  
    JZ FARE_BIRAKILDI
```

```
    CMP [old_bx], 1  
    JE SURE_KONTROL
```

```
    MOV [old_bx], 1
```

JMP KOORDINAT_AL

FARE_BIRAKILDI:

MOV [old_bx], 0
JMP SURE_KONTROL

KLAVYE_SECIM:

MOV AX, 03h
INT 33h

KOORDINAT_AL:

MOV AX, CX
MOV CL, 3
SHR AX, CL
MOV [click_x], AX

MOV AX, DX
MOV CL, 3
SHR AX, CL
MOV [click_y], AX

MOV SI, 0
JMP TIKLAMA_KONTROL

TIKLAMA_KONTROL:

CMP SI, NUM_OBJ
JGE SURE_KONTROL
CMP oact[SI], 0
JE SONRAKI_SAYI_TIK

MOV AX, [click_x]
MOV BL, ox[SI]
XOR BH, BH
SUB AX, BX
INC AX
CMP AX, 4
JA SONRAKI_SAYI_TIK

MOV AX, [click_y]
MOV BL, oy[SI]
XOR BH, BH
SUB AX, BX
INC AX
CMP AX, 2
JA SONRAKI_SAYI_TIK

VURULDU:

MOV DH, oy[SI]
MOV DL, ox[SI]
CALL SIL_2KARAKTER
MOV BX, SI
CALL SEL_OBJ
JMP SURE_KONTROL

SONRAKI_SAYI_TIK:

INC SI
JMP TIKLAMA_KONTROL

SURE_KONTROL:

MOV AH, 0
INT 1Ah
CMP DX, [last_tick]
JE HAREKET_KISMI
MOV [last_tick], DX

; --- DINAMIK MESAJ ZAMANLAYICI ---

```
CMP BYTE PTR [msg_timer], 0
JE TICK_DEVAM
DEC BYTE PTR [msg_timer]
CMP BYTE PTR [msg_timer], 0
JNE TICK_DEVAM
CALL CLEAR_MSG_LINE
TICK_DEVAM:
```

```
INC BYTE PTR [tick_cnt]
CMP BYTE PTR [tick_cnt], 1
JB HAREKET_KISMI
MOV BYTE PTR [tick_cnt], 0
```

```
DEC WORD PTR [timer]
CALL UPDATE_HUD
CMP WORD PTR [timer], 0
JNE HAREKET_KISMI
```

```
MOV BYTE PTR [gameover], 2
JMP END_SCREEN
```

```
HAREKET_KISMI:
CMP BYTE PTR [gameover], 0
JNE END_SCREEN
```

```
MOV AX, 02h
INT 33h
```

```
MOV SI, 0
CIZIM_DONGUSU:
CMP SI, NUM_OBJ
JGE BEKLEME_KISMI
```

```
CMP oact[SI], 0
JE SONRAKI_SAYI
```

```
MOV DH, oy[SI]
MOV DL, ox[SI]
CALL SIL_2KARAKTER
```

```
MOV AL, ox[SI]
ADD AL, vx[SI]
MOV ox[SI], AL
```

```
CMP AL, FIELD_LEFT
JLE X_YON_DEGISTIR_SAG
CMP AL, FIELD_RIGHT
JGE X_YON_DEGISTIR_SOL
JMP Y_GUNCELLE
X_YON_DEGISTIR_SAG:
MOV vx[SI], 1
JMP Y_GUNCELLE
X_YON_DEGISTIR_SOL:
MOV vx[SI], -1
```

```
Y_GUNCELLE:
MOV AL, oy[SI]
ADD AL, vy[SI]
MOV oy[SI], AL
```

```
CMP AL, 4
JLE Y_YON_DEGISTIR_ASAGI
CMP AL, 19
JGE Y_YON_DEGISTIR_YUKARI
JMP EKRANA_YAZ
Y_YON_DEGISTIR_ASAGI:
```

```
    MOV vy[SI], 1
    JMP EKRANA_YAZ
Y_YON_DEGISTIR_YUKARI:
    MOV vy[SI], -1
```

```
EKRANA_YAZ:
    MOV DH, oy[SI]
    MOV DL, ox[SI]
    CALL YAZ_2KARAKTER
```

```
SONRAKI_SAYI:
    INC SI
    JMP CIZIM_DONGUSU
```

```
BEKLEME_KISMI:
    MOV AX, 01h
    INT 33h
```

```
    CALL COLL_CHK
    CALL MIN_CHK
    CALL BEKLE
```

```
    JMP ANA_DONGU
```

```
FORCE_END:
    MOV BYTE PTR [gameover], 2
```

```
END_SCREEN:
    MOV AX, 02h
    INT 33h
```

```
    CALL DRAW_END
```

```
    MOV AX, 01h
    INT 33h
```

```
FLUSH_KBD:
    MOV AH, 01h
    INT 16h
    JZ WAIT_AGAIN
    MOV AH, 00h
    INT 16h
    JMP FLUSH_KBD
```

```
WAIT_AGAIN:
    MOV AH, 00h
    INT 16h
```

```
    CMP AL, 'A'
    JB WA_CMP
    CMP AL, 'Z'
    JA WA_CMP
    ADD AL, 20h
```

```
WA_CMP:
    CMP AL, 'r'
    JE DO_RESTART
    CMP AL, 'q'
    JE DO_EXIT
    JMP WAIT_AGAIN
```

```
DO_RESTART:
    CALL CLEAR_SCR
    CALL DRAW_STATIC
    CALL INIT_GAME
    CALL DRAW_ALL_NUMS
    JMP ANA_DONGU
```

```
DO_EXIT:
    MOV AX, 2
    INT 33h
    CALL CLEAR_SCR
    MOV AH, 4Ch
    INT 21h
```

```
; =====
; ALT PROGRAMLAR
; =====
```

```
DRAW_ALL_NUMS PROC
```

```
    PUSH SI
    PUSH AX
    MOV AX, 02h
    INT 33h
    MOV SI, 0
```

```
DAN_LOOP:
```

```
    CMP SI, NUM_OBJ
    JGE DAN_DONE
    CMP oact[SI], 0
    JE DAN_NEXT
    MOV DH, oy[SI]
    MOV DL, ox[SI]
    CALL YAZ_2KARAKTER
```

```
DAN_NEXT:
```

```
    INC SI
    JMP DAN_LOOP
```

```
DAN_DONE:
```

```
    MOV AX, 01h
    INT 33h
    POP AX
    POP SI
    RET
```

```
DRAW_ALL_NUMS ENDP
```

```
SIL_2KARAKTER PROC
```

```
    PUSH AX
    PUSH BX
    PUSH CX
    MOV AH, 02h
    MOV BH, 0
    INT 10h
    MOV AH, 09h
    MOV AL, ' '
    MOV BH, 0
    MOV BL, 07h
    MOV CX, 2
    INT 10h
    POP CX
    POP BX
    POP AX
    RET
```

```
SIL_2KARAKTER ENDP
```

```
YAZ_2KARAKTER PROC
```

```
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DI
    PUSH DX
```

```
    MOV AH, 02h
    MOV BH, 0
    INT 10h
```

```
MOV AX, SI
SHL AX, 1
MOV DI, AX
```

```
MOV AH, 09h
MOV AL, obj_chars[DI]
MOV BH, 0
MOV BL, obj_colors[SI]
MOV CX, 1
INT 10h
```

```
POP DX
PUSH DX
INC DL
MOV AH, 02h
MOV BH, 0
INT 10h
```

```
MOV AH, 09h
MOV AL, obj_chars[DI+1]
MOV BH, 0
MOV BL, obj_colors[SI]
MOV CX, 1
INT 10h
```

```
POP DX
POP DI
POP CX
POP BX
POP AX
RET
YAZ_2KARAKTER ENDP
```

```
BEKLE PROC
PUSH CX
PUSH DX
PUSH AX
MOV AH, 86h
MOV CX, [delay_hi]
MOV DX, [delay_lo]
INT 15h
POP AX
POP DX
POP CX
RET
BEKLE ENDP
```

```
UPDATE_STRINGS PROC
PUSH AX
PUSH BX
PUSH CX
PUSH DI
PUSH SI
```

```
MOV SI, 0
US_LOOP:
CMP SI, NUM_OBJ
JGE US_DONE
```

```
MOV AL, oval[SI]
MOV CX, SI
SHL CX, 1
MOV DI, CX
```

```
CMP AL, 0
JGE US_POS
```

```

MOV obj_chars[DI], '-'
NEG AL
ADD AL, '0'
MOV obj_chars[DI+1], AL
JMP US_NEXT

```

```

US_POS:
CMP AL, 10
JL US_SINGLE
MOV AH, 0
MOV BL, 10
DIV BL
ADD AL, '0'
ADD AH, '0'
MOV obj_chars[DI], AL
MOV obj_chars[DI+1], AH
JMP US_NEXT

```

```

US_SINGLE:
MOV obj_chars[DI], ' '
ADD AL, '0'
MOV obj_chars[DI+1], AL

```

```

US_NEXT:
INC SI
JMP US_LOOP

```

```

US_DONE:
POP SI
POP DI
POP CX
POP BX
POP AX
RET
UPDATE_STRINGS ENDP

```

```

; =====
; INIT_GAME & HUD GUNCELLEME
; =====
INIT_GAME PROC
MOV WORD PTR [score], 0
MOV WORD PTR [target], 15
MOV WORD PTR [current], 0
MOV BYTE PTR [level], 1
MOV WORD PTR [timer], TIMER_INIT
MOV BYTE PTR [gameover], 0

MOV BYTE PTR [tick_cnt], 0
MOV WORD PTR [last_tick], 0
MOV BYTE PTR [msg_timer], 0

MOV WORD PTR [delay_lo], 5000h
MOV WORD PTR [delay_hi], 0000h

MOV SI, 0
IG_LOOP:
CMP SI, NUM_OBJ
JAE IG_DONE
CALL RAND_X
MOV ox[SI], AL
CALL RAND_Y
MOV oy[SI], AL

CALL RAND_VEL
MOV vx[SI], AL

```

```

CALL RAND_VEL
MOV vy[SI], AL

CALL RAND_AL
MOV oval[SI], AL
MOV oact[SI], 1
INC SI
JMP IG_LOOP
IG_DONE:
CALL UPDATE_STRINGS
CALL UPDATE_HUD
RET
INIT_GAME ENDP

UPDATE_HUD PROC
PUSH AX
PUSH BX
PUSH CX
PUSH DX
PUSH SI

MOV AX, 02h
INT 33h

; --- SKOR ---
MOV DH, 2
MOV DL, 1
CALL GXY
MOV BL, 0Eh
MOV SI, OFFSET I_score
CALL PRT_STR_COLOR
MOV AX, [score]
CALL PRT_NUM_COLOR
MOV AL, ' '
CALL PRT_CHAR_COLOR
CALL PRT_CHAR_COLOR
CALL PRT_CHAR_COLOR

; --- HEDEF ---
MOV DH, 2
MOV DL, 16
CALL GXY
MOV BL, 0Dh
MOV SI, OFFSET I_target
CALL PRT_STR_COLOR
MOV AX, [target]
CALL PRT_NUM_COLOR
MOV AL, ' '
CALL PRT_CHAR_COLOR
CALL PRT_CHAR_COLOR
CALL PRT_CHAR_COLOR

; --- TOPLAM ---
MOV DH, 2
MOV DL, 31
CALL GXY
MOV BL, 0Bh
MOV SI, OFFSET I_current
CALL PRT_STR_COLOR
MOV AX, [current]
CALL PRT_NUM_COLOR
MOV AL, ' '
CALL PRT_CHAR_COLOR
CALL PRT_CHAR_COLOR
CALL PRT_CHAR_COLOR

```



```

; --- LEVEL ---
MOV DH, 2
MOV DL, 48
CALL GXY
MOV BL, 0Ah
MOV SI, OFFSET I_level
CALL PRT_STR_COLOR
MOV AL, [level]
XOR AH, AH
CALL PRT_NUM_COLOR
MOV AL, ' '
CALL PRT_CHAR_COLOR
CALL PRT_CHAR_COLOR

; --- SURE ---
MOV DH, 2
MOV DL, 60
CALL GXY
MOV BL, 0Dh
MOV SI, OFFSET I_timer
CALL PRT_STR_COLOR

MOV AX, [timer]
PUSH BX
XOR DX, DX
MOV BX, 60
DIV BX
POP BX

PUSH DX
CALL PRT_NUM_PAD2_COLOR

MOV AL, ':'
CALL PRT_CHAR_COLOR

POP AX
CALL PRT_NUM_PAD2_COLOR

MOV AL, ' '
CALL PRT_CHAR_COLOR
CALL PRT_CHAR_COLOR

MOV AX, 01h
INT 33h

POP SI
POP DX
POP CX
POP BX
POP AX
RET
UPDATE_HUD ENDP

PRT_CHAR_COLOR PROC
PUSH AX
PUSH BX
PUSH CX
MOV AH, 09h
MOV BH, 0
MOV CX, 1
INT 10h
MOV AH, 0Eh
INT 10h
POP CX
POP BX
POP AX

```

```
    RET
PRT_CHAR_COLOR ENDP
```

```
PRT_STR_COLOR PROC
```

```
    PUSH AX
```

```
    PUSH BX
```

```
    PUSH CX
```

```
PSC_LOOP:
```

```
    LODSB
```

```
    CMP AL, '$'
```

```
    JE PSC_DONE
```

```
    MOV AH, 09h
```

```
    MOV BH, 0
```

```
    MOV CX, 1
```

```
    INT 10h
```

```
    MOV AH, 0Eh
```

```
    INT 10h
```

```
    JMP PSC_LOOP
```

```
PSC_DONE:
```

```
    POP CX
```

```
    POP BX
```

```
    POP AX
```

```
    RET
```

```
PRT_STR_COLOR ENDP
```

```
PRT_NUM_COLOR PROC
```

```
    PUSH AX
```

```
    PUSH BX
```

```
    PUSH CX
```

```
    PUSH DX
```

```
    MOV CX, 0
```

```
    OR AX, AX
```

```
    JNS PNC_POS
```

```
    PUSH AX
```

```
    MOV AL, '-'
```

```
    CALL PRT_CHAR_COLOR
```

```
    POP AX
```

```
    NEG AX
```

```
PNC_POS:
```

```
    CMP AX, 0
```

```
    JNE PNC_DIGITS
```

```
    MOV AL, '0'
```

```
    CALL PRT_CHAR_COLOR
```

```
    JMP PNC_RET
```

```
PNC_DIGITS:
```

```
    MOV CX, 0
```

```
PNC_PUSH:
```

```
    CMP AX, 0
```

```
    JE PNC_POP
```

```
    XOR DX, DX
```

```
    PUSH BX
```

```
    MOV BX, 10
```

```
    DIV BX
```

```
    POP BX
```

```
    ADD DL, '0'
```

```
    PUSH DX
```

```
    INC CX
```

```
    JMP PNC_PUSH
```

```
PNC_POP:
```

```
    CMP CX, 0
```

```
    JE PNC_RET
```

```
    POP AX
```

```
    CALL PRT_CHAR_COLOR
```

```
DEC CX
JMP PNC_POP
```

```
PNC_RET:
POP DX
POP CX
POP BX
POP AX
RET
PRT_NUM_COLOR ENDP
```

```
PRT_NUM_PAD2_COLOR PROC
PUSH AX
PUSH BX
PUSH CX
PUSH DX
```

```
XOR DX, DX
PUSH BX
MOV BX, 10
DIV BX
POP BX
```

```
PUSH DX
ADD AL, '0'
CALL PRT_CHAR_COLOR
POP AX
ADD AL, '0'
CALL PRT_CHAR_COLOR
```

```
POP DX
POP CX
POP BX
POP AX
RET
PRT_NUM_PAD2_COLOR ENDP
```

```
; =====
; DINAMIK MESAJ GOSTERIMI
; =====
```

```
SHOW_MSG PROC
```

```
PUSH AX
PUSH BX
PUSH CX
PUSH DX
PUSH SI
```

```
MOV SI, DX
```

```
MOV DH, 22
MOV DL, 1
CALL GXY
```

```
MOV AH, 09h
MOV AL, ' '
MOV BL, CL
MOV BH, 0
PUSH CX
MOV CX, 70
INT 10h
POP CX
```

```
MOV DX, SI
MOV AH, 09h
INT 21h
```

```
MOV BYTE PTR [msg_timer], 36
```

```
POP SI
POP DX
POP CX
POP BX
POP AX
RET
SHOW_MSG ENDP
```

```
CLEAR_MSG_LINE PROC
```

```
PUSH AX
PUSH BX
PUSH CX
PUSH DX
MOV DH, 22
MOV DL, 1
CALL GXY
MOV AH, 09h
MOV AL, ' '
MOV BH, 0
MOV BL, 07h
MOV CX, 70
INT 10h
POP DX
POP CX
POP BX
POP AX
RET
```

```
CLEAR_MSG_LINE ENDP
```

```
; =====
; SECIM VE YENI SEVIYE
; =====
```

```
SEL_OBJ PROC
```

```
PUSH AX
PUSH CX
PUSH DX
```

```
CMP BYTE PTR oact[BX], 0
JE SELOBJ_DONE
```

```
MOV AL, oval[BX]
CBW
ADD [current], AX
```

```
MOV AL, oval[BX]
OR AL, AL
JS NEG_CASE
```

```
CALL CHK_PRIME
CMP AH, 1
JNE NORMAL_SCORE
ADD WORD PTR [score], 10
MOV CX, 1
JMP AFTER_SCORE
```

```
NORMAL_SCORE:
```

```
MOV CX, 0
JMP AFTER_SCORE
```

```
NEG_CASE:
```

```
SUB WORD PTR [score], 10
MOV CX, 2
```

```
AFTER_SCORE:
```

```
MOV BYTE PTR oact[BX], 0
CALL UPDATE_HUD
```

```
MOV AX, [current]
CMP AX, [target]
JE MATCH_TARGET
JMP CHECK_NEG_PRIME
```

```
MATCH_TARGET:
CALL DO_NEXT_LVL
JMP SELOBJ_DONE
```

```
CHECK_NEG_PRIME:
CMP CX, 1
JNE CHECK_NEG
MOV CX, 0Dh ; Pembe
MOV DX, OFFSET msg_prime
CALL SHOW_MSG
JMP SELOBJ_DONE
```

```
CHECK_NEG:
CMP CX, 2
JNE SELOBJ_DONE
MOV CX, 0Ch ; Kirmizi
MOV DX, OFFSET msg_neg
CALL SHOW_MSG
```

```
SELOBJ_DONE:
POP DX
POP CX
POP AX
RET
SEL_OBJ ENDP
```

```
DO_NEXT_LVL PROC
ADD WORD PTR [score], 50
INC BYTE PTR [level]
ADD WORD PTR [target], 10
MOV WORD PTR [current], 0
MOV WORD PTR [timer], TIMER_INIT
```

```
CALL RESET_ALL_NUMS
```

```
MOV AX, [delay_lo]
SUB AX, 1000h
CMP AX, 2000h
JG DNL_SAVE
MOV AX, 2000h
```

```
DNL_SAVE:
MOV [delay_lo], AX
```

```
MOV CX, 0Ah ; Yesil
MOV DX, OFFSET msg_newlevel
CALL SHOW_MSG
```

```
RET
DO_NEXT_LVL ENDP
```

```
; =====
; COLL_CHK - ÇARPIŞMA KONTROLÜ
; =====
```

```
COLL_CHK PROC
PUSH AX
PUSH BX
PUSH CX
PUSH SI
```

```

PUSH DI

MOV SI, 0
CC_OUTER:
CMP SI, 9
JAE CC_DONE
CMP BYTE PTR oact[SI], 0
JE CC_NEXTI

MOV DI, SI
INC DI
CC_INNER:
CMP DI, NUM_OBJ
JAE CC_NEXTI
CMP BYTE PTR oact[DI], 0
JE CC_NEXTJ

MOV AL, ox[SI]
MOV BL, ox[DI]
SUB AL, BL
JNS CC_POSX
NEG AL
CC_POSX:
CMP AL, 4
JA CC_NEXTJ

MOV AL, oy[SI]
MOV BL, oy[DI]
SUB AL, BL
JNS CC_POSY
NEG AL
CC_POSY:
CMP AL, 2
JA CC_NEXTJ

MOV BYTE PTR oact[SI], 0
MOV DH, oy[SI]
MOV DL, ox[SI]
CALL SIL_2KARAKTER

MOV BYTE PTR oact[DI], 0
MOV DH, oy[DI]
MOV DL, ox[DI]
CALL SIL_2KARAKTER

MOV CX, 0Eh ; Sari
MOV DX, OFFSET msg_crash
CALL SHOW_MSG

CC_NEXTJ:
INC DI
JMP CC_INNER
CC_NEXTI:
INC SI
JMP CC_OUTER

CC_DONE:
POP DI
POP SI
POP CX
POP BX
POP AX
RET
COLL_CHK ENDP

MIN_CHK PROC

```

```

    PUSH AX
    PUSH BX
    MOV BX, 0
    MOV AL, 0
MC_CNT:
    CMP BX, NUM_OBJ
    JAE MC_CHK
    CMP BYTE PTR oact[BX], 1
    JNE MC_SKIP
    INC AL
MC_SKIP:
    INC BX
    JMP MC_CNT
MC_CHK:
    CMP AL, NUM_OBJ
    JAE MC_DONE

    MOV BX, 0
MC_RVIVE:
    CMP BX, NUM_OBJ
    JAE MC_DONE
    CMP BYTE PTR oact[BX], 0
    JNE MC_RNXT
    MOV BYTE PTR oact[BX], 1

    CALL RAND_X
    MOV ox[BX], AL
    CALL RAND_Y
    MOV oy[BX], AL
    CALL RAND_VEL
    MOV vx[BX], AL
    CALL RAND_VEL
    MOV vy[BX], AL
    CALL RAND_AL
    MOV oval[BX], AL

MC_RNXT:
    INC BX
    JMP MC_RVIVE

MC_DONE:
    CALL UPDATE_STRINGS
    POP BX
    POP AX
    RET
MIN_CHK ENDP

RESET_ALL_NUMS PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH SI

    MOV AX, 02h
    INT 33h

    MOV SI, 0
RESET_LOOP:
    CMP SI, NUM_OBJ
    JAE RESET_DONE

    CALL RAND_X
    MOV ox[SI], AL
    CALL RAND_Y
    MOV oy[SI], AL

```

```
CALL RAND_VEL
MOV vx[SI], AL
```

```
CALL RAND_VEL
MOV vy[SI], AL
```

```
CALL RAND_AL
MOV oval[SI], AL
MOV oact[SI], 1
```

```
INC SI
JMP RESET_LOOP
```

```
RESET_DONE:
CALL UPDATE_STRINGS
CALL CLEAR_SCR
CALL DRAW_STATIC
CALL DRAW_ALL_NUMS
CALL UPDATE_HUD
```

```
MOV AX, 01h
INT 33h
```

```
POP SI
POP CX
POP BX
POP AX
RET
```

```
RESET_ALL_NUMS ENDP
```

```
; =====
; DRAW_STATIC & MENU EKRANLARI
; =====
```

```
DRAW_STATIC PROC
```

```
MOV DH, 0
MOV DL, 0
CALL GXY
MOV CX, 80
CALL RENK_PEMBE_ZEMIN
MOV DH, 0
MOV DL, 12
CALL GXY
MOV DX, OFFSET s_title
MOV AH, 09h
INT 21h
```

```
MOV DH, 1
MOV DL, 0
CALL GXY
MOV CX, 80
CALL RENK_PEMBE
MOV DX, OFFSET s_line
MOV AH, 09h
INT 21h
```

```
MOV DH, 3
MOV DL, 0
CALL GXY
MOV CX, 80
CALL RENK_PEMBE
MOV DX, OFFSET s_line
MOV AH, 09h
INT 21h
```

```
MOV DH, 20
MOV DL, 0
```



```
CALL GXY
MOV CX, 80
CALL RENK_PEMBE
MOV DX, OFFSET s_line
MOV AH, 09h
INT 21h
```

```
MOV DH, 21
MOV DL, 0
CALL GXY
MOV CX, 80
CALL RENK_MAVI_ZEMIN
MOV DH, 21
MOV DL, 1
CALL GXY
MOV DX, OFFSET s_keys
MOV AH, 09h
INT 21h
```

```
RET
DRAW_STATIC ENDP
```

```
DRAW_END PROC
CALL CLEAR_SCR
MOV DH, 8
MOV DL, 15
CALL GXY
CMP BYTE PTR [gameover], 1
JNE DE_LOSE
MOV CX, 40
CALL RENK_YESIL
MOV DX, OFFSET m_win
JMP DE_SHOW
```

```
DE_LOSE:
MOV CX, 40
CALL RENK_KIRMIZI
MOV DX, OFFSET m_lose
```

```
DE_SHOW:
MOV AH, 09h
INT 21h
```

```
MOV DH, 10
MOV DL, 15
CALL GXY
MOV BL, 0Eh
MOV SI, OFFSET I_score
CALL PRT_STR_COLOR
MOV AX, [score]
CALL PRT_NUM_COLOR
```

```
MOV DH, 12
MOV DL, 12
CALL GXY
MOV CX, 50
CALL RENK_MAVI_ZEMIN
MOV DX, OFFSET m_again
MOV AH, 09h
INT 21h
RET
DRAW_END ENDP
```

```
SHOW_HELP PROC
CALL CLEAR_SCR
```

```
MOV DH, 1
MOV DL, 18
```

```
CALL GXY
MOV CX, 46
CALL RENK_PEMBE_ZEMIN
MOV DX, OFFSET h0
MOV AH, 09h
INT 21h
```

```
MOV DH, 2
MOV DL, 18
CALL GXY
MOV CX, 46
CALL RENK_PEMBE_ZEMIN
MOV DX, OFFSET h00
MOV AH, 09h
INT 21h
```

```
MOV DH, 4
MOV DL, 3
CALL GXY
MOV CX, 70
CALL RENK_SARI
MOV DX, OFFSET h1
MOV AH, 09h
INT 21h
```

```
MOV DH, 5
MOV DL, 3
CALL GXY
MOV CX, 70
CALL RENK_BEYAZ
MOV DX, OFFSET h2
MOV AH, 09h
INT 21h
```

```
MOV DH, 6
MOV DL, 3
CALL GXY
MOV CX, 70
CALL RENK_BEYAZ
MOV DX, OFFSET h3
MOV AH, 09h
INT 21h
```

```
MOV DH, 9
MOV DL, 3
CALL GXY
MOV CX, 70
CALL RENK_CYAN
MOV DX, OFFSET h5
MOV AH, 09h
INT 21h
```

```
MOV DH, 10
MOV DL, 3
CALL GXY
MOV CX, 70
CALL RENK_BEYAZ
MOV DX, OFFSET h6
MOV AH, 09h
INT 21h
```

```
MOV DH, 11
MOV DL, 3
CALL GXY
MOV CX, 70
CALL RENK_BEYAZ
```

```
MOV DX, OFFSET h7
MOV AH, 09h
INT 21h
```

```
MOV DH, 13
MOV DL, 3
CALL GXY
MOV CX, 70
CALL RENK_YESIL
MOV DX, OFFSET h8
MOV AH, 09h
INT 21h
```

```
MOV DH, 14
MOV DL, 3
CALL GXY
MOV CX, 70
CALL RENK_BEYAZ
MOV DX, OFFSET h9
MOV AH, 09h
INT 21h
```

```
MOV DH, 15
MOV DL, 3
CALL GXY
MOV CX, 70
CALL RENK_BEYAZ
MOV DX, OFFSET h10
MOV AH, 09h
INT 21h
```

```
MOV DH, 16
MOV DL, 3
CALL GXY
MOV CX, 70
CALL RENK_KIRMIZI
MOV DX, OFFSET h11
MOV AH, 09h
INT 21h
```

```
MOV DH, 19
MOV DL, 15
CALL GXY
MOV CX, 50
CALL RENK_PEMBE
MOV DX, OFFSET hstart
MOV AH, 09h
INT 21h
```

```
RET
SHOW_HELP ENDP
```

```
; =====
; RASTGELE METOTLAR
; =====
RAND_X PROC
    PUSH BX
    PUSH DX
    MOV AX, [rseed]
    MOV BX, 25173
    MUL BX
    ADD AX, 13849
    MOV [rseed], AX
    MOV AH, 0
    MOV BL, 60
    DIV BL
```

```
    MOV AL, AH
    ADD AL, 5
    POP DX
    POP BX
    RET
RAND_X ENDP
```

```
RAND_Y PROC
    PUSH BX
    PUSH DX
    MOV AX, [rseed]
    MOV BX, 25173
    MUL BX
    ADD AX, 13849
    MOV [rseed], AX
    MOV AH, 0
    MOV BL, 14
    DIV BL
    MOV AL, AH
    ADD AL, 5
    POP DX
    POP BX
    RET
RAND_Y ENDP
```

```
RAND_AL PROC
    PUSH BX
    PUSH DX
    MOV AX, [rseed]
    MOV BX, 25173
    MUL BX
    ADD AX, 13849
    MOV [rseed], AX
    MOV AH, 0
    MOV BL, 35
    DIV BL
    MOV AL, AH
    SUB AL, 9
    POP DX
    POP BX
    RET
RAND_AL ENDP
```

```
RAND_VEL PROC
    PUSH BX
    PUSH DX
    MOV AX, [rseed]
    MOV BX, 25173
    MUL BX
    ADD AX, 13849
    MOV [rseed], AX
    MOV AH, 0
    MOV BL, 2
    DIV BL
    CMP AH, 0
    JE RV_NEG
    MOV AL, 1
    JMP RV_DONE
RV_NEG:
    MOV AL, -1
RV_DONE:
    POP DX
    POP BX
    RET
RAND_VEL ENDP
```

```

; =====
; YARDIMCI METOTLAR
; =====
CHK_PRIME PROC
    MOV AH, 0
    CMP AL, 2
    JE CP_YES
    CMP AL, 3
    JE CP_YES
    CMP AL, 5
    JE CP_YES
    CMP AL, 7
    JE CP_YES
    CMP AL, 11
    JE CP_YES
    CMP AL, 13
    JE CP_YES
    CMP AL, 17
    JE CP_YES
    CMP AL, 19
    JE CP_YES
    CMP AL, 23
    JE CP_YES
    RET
CP_YES:
    MOV AH, 1
    RET
CHK_PRIME ENDP

PRT_NUM PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX
    OR AX, AX
    JNS PN_NONZERO_CHK
    PUSH AX
    MOV AH, 02h
    MOV DL, '-'
    INT 21h
    POP AX
    NEG AX
PN_NONZERO_CHK:
    CMP AX, 0
    JNE PN_NONZERO
    MOV AH, 02h
    MOV DL, '0'
    INT 21h
    JMP PN_RET
PN_NONZERO:
    MOV BX, 0
PN_PUSH:
    CMP AX, 0
    JE PN_POP
    MOV DX, 0
    MOV CX, 10
    DIV CX
    ADD DL, '0'
    PUSH DX
    INC BX
    JMP PN_PUSH
PN_POP:
    CMP BX, 0
    JE PN_RET
    POP DX
    MOV AH, 02h

```

```
    INT 21h
    DEC BX
    JMP PN_POP
PN_RET:
    POP DX
    POP CX
    POP BX
    POP AX
    RET
PRT_NUM ENDP
```

```
PRT_NUM_PAD2 PROC
    PUSH AX
    PUSH BX
    PUSH DX
```

```
    MOV BL, 10
    DIV BL
```

```
    MOV DL, AL
    ADD DL, '0'
    PUSH AX
    MOV AH, 02h
    INT 21h
    POP AX
```

```
    MOV DL, AH
    ADD DL, '0'
    MOV AH, 02h
    INT 21h
```

```
    POP DX
    POP BX
    POP AX
    RET
```

```
PRT_NUM_PAD2 ENDP
```

```
GXY PROC
    PUSH AX
    PUSH BX
    MOV AH, 02h
    MOV BH, 0
    INT 10h
    POP BX
    POP AX
    RET
```

```
GXY ENDP
```

```
CLEAR_SCR PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX
    MOV AH, 06h
    MOV AL, 0
    MOV BH, 07h
    MOV CX, 0
    MOV DX, 184Fh
    INT 10h
    MOV DH, 0
    MOV DL, 0
    CALL GXY
    POP DX
    POP CX
    POP BX
    POP AX
```

```
    RET
CLEAR_SCR ENDP

RENK_PEMBE PROC
    PUSH AX
    PUSH BX
    MOV AH, 09h
    MOV AL, ' '
    MOV BH, 0
    MOV BL, 0Dh
    INT 10h
    POP BX
    POP AX
    RET
RENK_PEMBE ENDP

RENK_SARI PROC
    PUSH AX
    PUSH BX
    MOV AH, 09h
    MOV AL, ' '
    MOV BH, 0
    MOV BL, 0Eh
    INT 10h
    POP BX
    POP AX
    RET
RENK_SARI ENDP

RENK_CYAN PROC
    PUSH AX
    PUSH BX
    MOV AH, 09h
    MOV AL, ' '
    MOV BH, 0
    MOV BL, 0Bh
    INT 10h
    POP BX
    POP AX
    RET
RENK_CYAN ENDP

RENK_YESIL PROC
    PUSH AX
    PUSH BX
    MOV AH, 09h
    MOV AL, ' '
    MOV BH, 0
    MOV BL, 0Ah
    INT 10h
    POP BX
    POP AX
    RET
RENK_YESIL ENDP

RENK_KIRMIZI PROC
    PUSH AX
    PUSH BX
    MOV AH, 09h
    MOV AL, ' '
    MOV BH, 0
    MOV BL, 0Ch
    INT 10h
    POP BX
    POP AX
    RET
```

RENK_KIRMIZI ENDP

RENK_BEYAZ PROC

```
PUSH AX
PUSH BX
MOV AH, 09h
MOV AL, ' '
MOV BH, 0
MOV BL, 0Fh
INT 10h
POP BX
POP AX
RET
```

RENK_BEYAZ ENDP

RENK_PEMBE_ZEMIN PROC

```
PUSH AX
PUSH BX
MOV AH, 09h
MOV AL, ' '
MOV BH, 0
MOV BL, 5Dh
INT 10h
POP BX
POP AX
RET
```

RENK_PEMBE_ZEMIN ENDP

RENK_MAVI_ZEMIN PROC

```
PUSH AX
PUSH BX
MOV AH, 09h
MOV AL, ' '
MOV BH, 0
MOV BL, 1Dh
INT 10h
POP BX
POP AX
RET
```

RENK_MAVI_ZEMIN ENDP

```
; =====
; VERI ALANI
; =====
```

```
ox DB 10 DUP(0)
oy DB 10 DUP(0)
vx DB 10 DUP(0)
vy DB 10 DUP(0)
oval DB 10 DUP(0)
oact DB 10 DUP(0)
```

```
obj_chars DB 10 DUP(' ')
obj_colors DB 0Dh, 0Eh, 0Bh, 0Ah, 0Ch, 0Fh, 0Dh, 0Eh, 0Bh, 0Ah
```

```
score DW 0
target DW 15
current DW 0
level DB 1
timer DW TIMER_INIT
gameover DB 0
rseed DW 9173h
```

```
tick_cnt DB 0
last_tick DW 0
delay_lo DW 1000h
delay_hi DW 0000h
```



```
old_bx    DW 0
click_x   DW 0
click_y   DW 0
msg_timer DB 0
```

```
s_title DB '=== HEDEF SAYIYA ULAS - 8086 Assembly Oyunu ===$'
```

s_line DB

[illegible]

s_keys DB ' Fare Sol Tik VEYA Bosluk = Sec | [Q] = Cikis \$'

```
l_score DB 'SKOR: $'
```

```
l_target DB '| HEDEF: $'
```

I_current DB | TOPLAM: \$'

```
l_level DB '| LEVEL: $'
```

```
l_timer DB '| SURE: $'
```

```
m_win DB '*** TEBRIKLER! Hedefe ulastiniz! ***$'
```

```
m_lose    DB '*** !!! OYUN BITTI !!! ***$'
```

```
m_again DB '[R] = Tekrar Oyna | [Q] = Programdan Cik$'
```

```
msg_prime DB ' >> ASAL SAYI SECILDI +10 << $'
```

```
msg_neg      DB ' >> NEGATIF SAYI SECILDI -10 <<'
```

```
msg_newlevel DB ' >> HEDEF GECILDI +50 << $'
```

```
msg_crash    DB ' >> CARPISMA! Iki kutu yok oldu. <<                                $
```

DB ' ===== \$'

h00 DB ' HEDEF SAYIYA ULAS - NASIL OYNANIR? '\$'

```
h1 DB 'AMAC:$'
```

DB ' Ekranda duz sekilde ucan sayilara farenizle tiklayin.\$'

DB ' Sectiginiz sayilar TOPLAM a eklenir.\$'

h4 DB ' TOPLAM = HEDEF oldugunda level gecersiniz!\$'

```
h5 DB 'KONTROLLER:$'
```

DB ' Sol Tik / Bosluk --> Uzerine gelinen sayiyi secer\$'

h7 DB ' Q = Cıkıs (Oyun icindeyken)\$'

```
h8      DB 'PUANLAMA:$'
```

h9 DB ' Hedefe ulasmak : +50 puan ve Hiz Artisi\$'

h10 DB ' Asal sayi secmek : +10 puan bonus\$'

h11 DB ' Negatif sayi secmek : -10 puan ceza\$'

```
hstart DB '>>> Herhangi bir tusa basin... <<<$'
```

END START